

Sensay (Token)

SMART CONTRACT

Security Audit

Performed on Contracts:

Sensay.sol

MD5 Hash: 2e3406b7b4facf414b92b3cf0f9ccb11

Platform

ETH

hashlock.com.au

MAR 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Standardised Checks	9
Intended Smart Contract Functions	11
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Audit Findings	13
Centralisation	61
Conclusion	62
Our Methodology	63
Disclaimers	65
About Hashlock	66

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Sensay team partnered with Hashlock to conduct a security audit of their Sensay.sol smart contract token. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Sensay token is an ERC20 compatible token designed to facilitate transactions with the Sensay platform, access Sensay services, grow the platform, empower data ownership (such as leasing out their digital replicas) and give exclusive access to unique features for users.

The token also allows for staking where holders can stake \$SNSY to earn rewards over a period of time and participate in governance proposals. Rewards will be generated from the platform performance as well as over all growth.

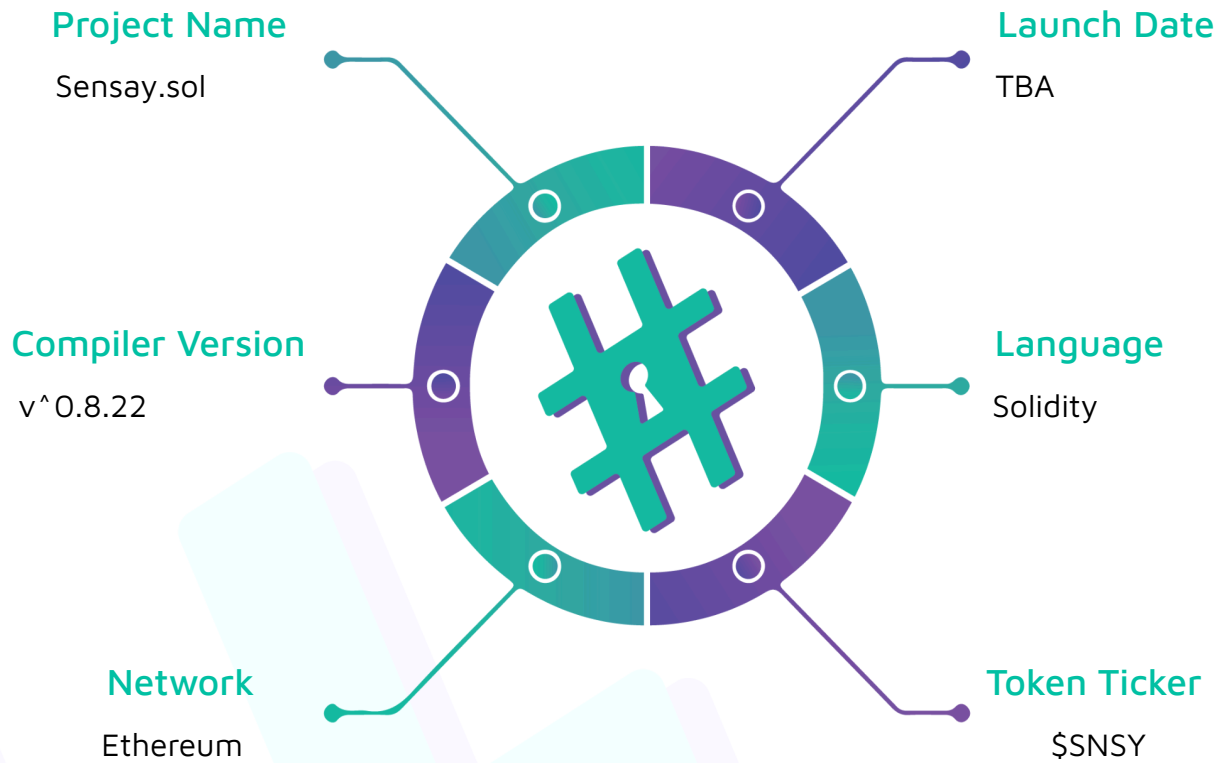
Project Name: Sensay.sol

Compiler Version: ^0.8.22

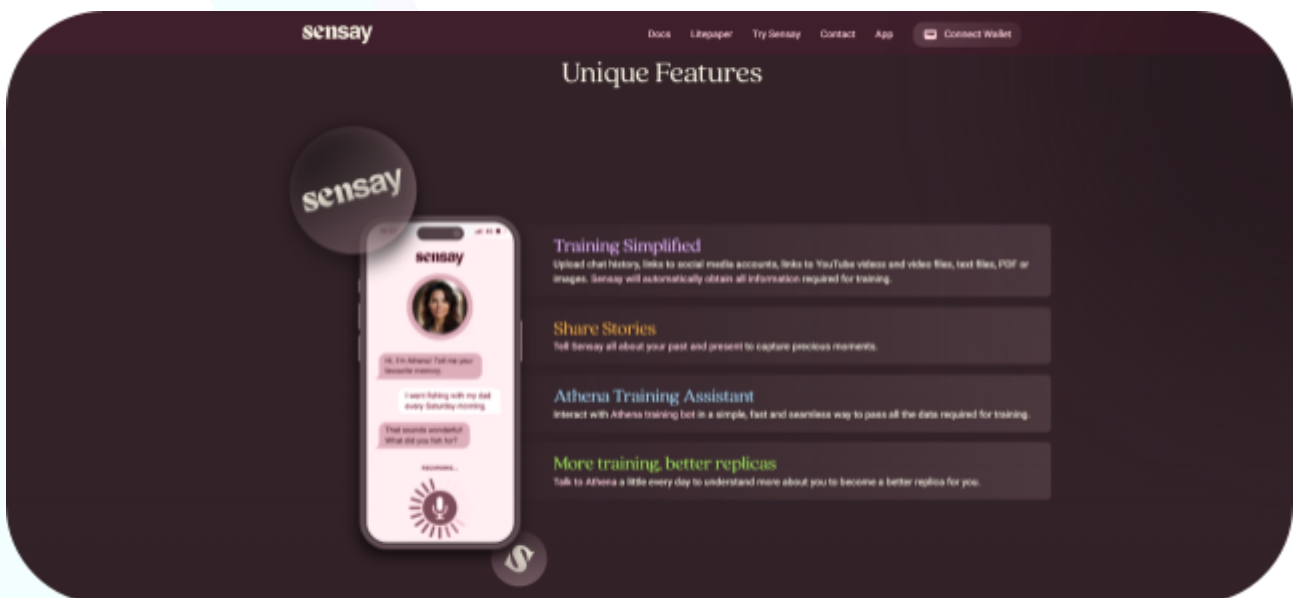
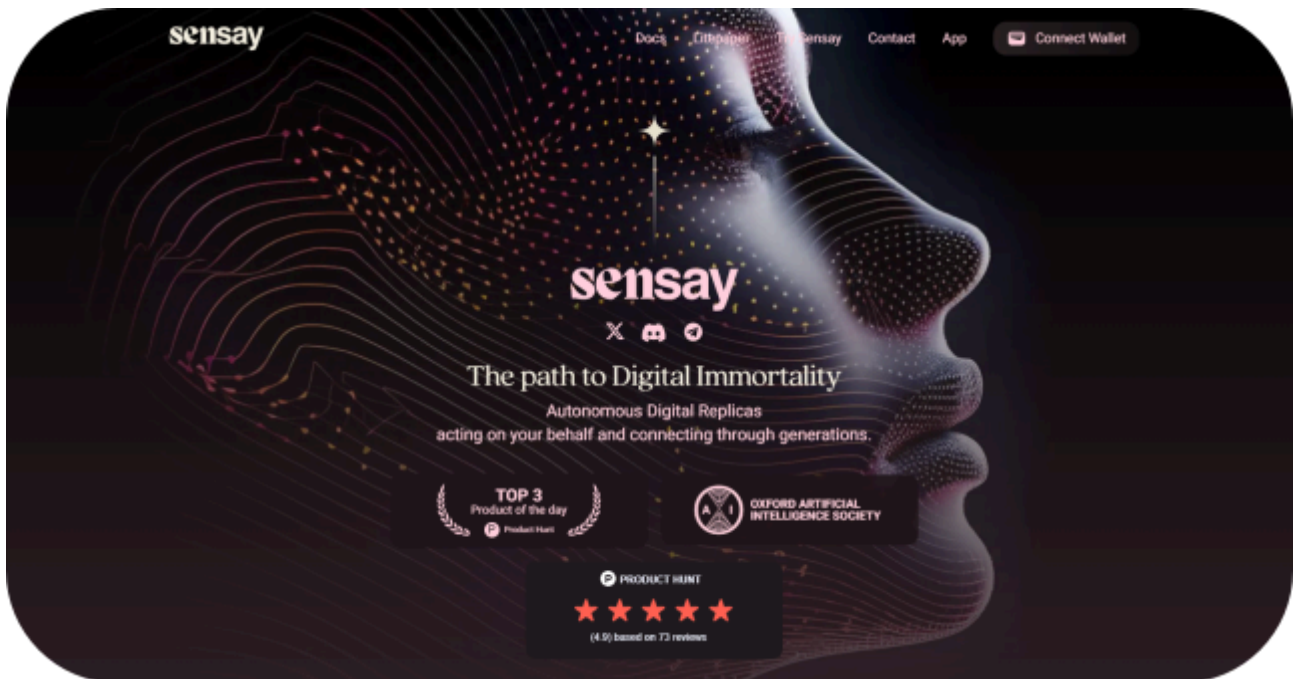
Website: <https://www.snsy.ai/> and <https://sensay.io/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Sensay Token project, the scope of works included a comprehensive review of the smart contract(s) listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Template Protocol Smart Contracts
Platform	Blast Network / Solidity
Audit Date	Mar, 2022
Contract 1	Sensay.sol
Contract 1 MD5 Hash	2e3406b7b4facf414b92b3cf0f9ccb11

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses the Ownable library from the Open Zeppelin contracts library as well as being built on LayerZero's OFT.sol library. With a simple implementation from Sensay and the usage of thoroughly audited and formally verified smart contract libraries, Sensay has achieved an optimal security standard.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. General security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

0 High severity vulnerabilities

0 Medium severity vulnerabilities

0 Low severity vulnerabilities

0 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*



Hashlock Pty Ltd

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Passed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Passed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed

	High consumption 'for/while' loop	Passed
	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: SECURE

Revised Audit Result: N/a

Intended Smart Contract Behaviour

Claimed Behaviour	Actual Behaviour
Sensay.sol - Mints the owner the initial token supply - Standard behaviour expected from a typical ERC20 token - The OFT (Omnichain Fungible Token) allows this token to be used across as many chains as possible. With auditing from LayerZero providers and formal verification from certora, this achieves the expected behaviour.	Contract achieves this functionality.

Code Quality

This Audit scope involves a smart contract from the Sensay project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Sensay project's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects. The code can be verified [here](#).

Apart from libraries, its implementation may be used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

No High severity vulnerabilities were identified.

Medium

No Medium severity vulnerabilities were identified.

Low

No Low severity vulnerabilities were identified.

Gas

No Gas optimisations were found.

Hacker's Notes

Sensay has taken a minimalistic but effective approach towards securing their token contract. Simplistic code in conjunction with battle tested libraries are necessary for achieving optimal security for smart contracts. The token contract makes use of LayerZero's OFT.sol and OpenZeppelin's Ownable.sol dependencies.

LayerZero labs has commissioned over 35 smart contract audits. In addition to this, nearly all the code written by LayerZero Labs since its birth have been immutable smart contracts audited externally and rigorously reviewed internally at least 3 times each. The below links contain smart contract audits for LayerZero's OFT.sol dependency:

- ChainSecurity OFT_OApp (Jan 2024):
https://github.com/LayerZero-Labs/Audits/blob/main/audits/ChainSecurity_LayerZero_OFT_OApp_audit.pdf
- Zellic OFT Wrapper Audit (Jan 2023):
[https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20OFT%20Wrapper%20Audit%20\(January%2019th%202023\)%20-%20Zellic.pdf](https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20OFT%20Wrapper%20Audit%20(January%2019th%202023)%20-%20Zellic.pdf)
- Zellic NativeOFTWithFee Audit (June 2023):
[https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20NativeOFTWithFee%20Audit%20\(June%2030th%202023\)%20-%20Zellic.pdf](https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20NativeOFTWithFee%20Audit%20(June%2030th%202023)%20-%20Zellic.pdf)
- Certora Formal Verification Report (July 2022):
[https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20Formal%20Verification%20Report%20\(July%2028th%202022\)%20-%20Certora.pdf](https://github.com/LayerZero-Labs/Audits/blob/main/audits/LayerZero%20Formal%20Verification%20Report%20(July%2028th%202022)%20-%20Certora.pdf)

In addition to the smart contract audits, both LayerZero and OpenZeppelin have both issued Bug Bounty programs on Immunefi with both dependencies in scope for the program:

- LayerZero Bug Bounty Program (Max Bounty: \$15M USDC)
<https://immunefi.com/bounty/layerzero/>
- OpenZeppelin Contracts Bug Bounty Program (Max Bounty: \$25,000 USDC)
<https://immunefi.com/bounty/openzeppelin/>

Centralisation

The Sensay project values security and utility over decentralisation.

The owner is minted the total supply of tokens within the protocol however, with simplistic implementation and usage of strong libraries, the token increases security and functionality but depends highly on the internal team to responsibly distribute tokens amongst the users.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Sensay project seems to have a sound codebase however, the repository itself is missing unit tests for sanity checking that the owner is minted the correct supply of tokens. Overall, most of the code is correctly ordered and follows industry best practices which includes battle tested libraries and a minimalistic, simple approach in the implementation to mitigate the chances of bugs occurring. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd